Practical 11

Problem definition:

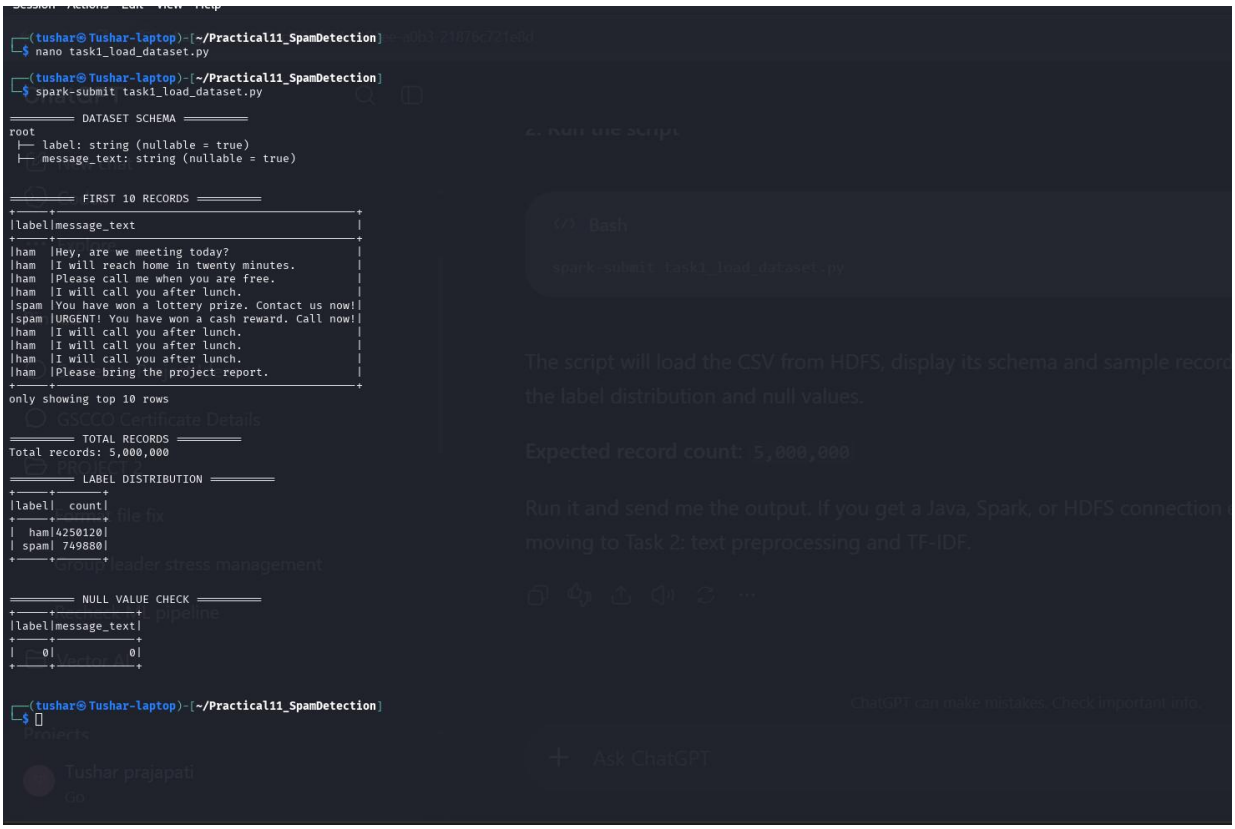
You are a Big Data Engineer at SecureComm Inc., a telecom company that processes over 50 million SMS messages daily. The fraud prevention team wants an automated spam classifier that can scale to handle millions of new messages per hour. You will implement a Distributed Logistic Regression model using Mini-batch Gradient Descent on Apache Spark, train it on a massive SMS dataset, and deploy it for batch prediction.

Objectives:

1. To load a large-scale SMS spam dataset from HDFS into a Spark DataFrame.
2. To preprocess text data using tokenization, stop-word removal, and TF-IDF feature extraction.
3. To implement Logistic Regression using Spark and mini-batch gradient descent.
4. To tune hyperparameters such as regParam, maxIter, and elasticNetParam.
5. To evaluate the classifier using Accuracy, Precision, Recall, F1-score, and ROC-AUC.
6. To analyze the effect of increasing dataset size on training time and model accuracy.
7. To compare the performance of full-batch and mini-batch gradient descent.

Observation / Output:

Task 1 : Load the large-scale SMS spam dataset from HDFS into a Spark DataFrame.



```
(tushar㉿Tushar-laptop)-[~/Practical11_SpamDetection]
$ nano task1_load_dataset.py

(tushar㉿Tushar-laptop)-[~/Practical11_SpamDetection]
$ spark-submit task1_load_dataset.py

========== DATASET SCHEMA ==========
root
 ├── label: string (nullable = true)
 ├── message_text: string (nullable = true)


========== FIRST 10 RECORDS ==========
+-----+--------------------------------------------+
|label|message_text                                |
+-----+--------------------------------------------+
|ham  |Hey, are we meeting today?                  |
|ham  |I will reach home in twenty minutes.        |
|ham  |Please call me when you are free.           |
|ham  |I will call you after lunch.                |
|spam |You have won a lottery prize. Contact us now!|
|spam |URGENT! You have won a cash reward. Call now!|
|ham  |I will call you after lunch.                |
|ham  |I will call you after lunch.                |
|ham  |I will call you after lunch.                |
|ham  |Please bring the project report.            |
+-----+--------------------------------------------+
only showing top 10 rows

========== TOTAL RECORDS ==========
Total records: 5,000,000

========== LABEL DISTRIBUTION ==========
+-----+-------+
|label|  count|
+-----+-------+
|  ham|4250120|
| spam| 749880|
+-----+-------+


========== NULL VALUE CHECK ==========
+-----+------------+
|label|message_text|
+-----+------------+
|    0|           0|
+-----+------------+


(tushar㉿Tushar-laptop)-[~/Practical11_SpamDetection]
$
```

Task 2: Perform text preprocessing: tokenization, stop word removal, and TF-IDF feature extraction.



```
(tushar㉿Tushar-laptop)-[~/Practical11_SpamDetection]
$ spark-submit task2_preprocessing.py
======== ORIGINAL DATA ========
+-----+--------------------+
|label|message_text        |
+-----+--------------------+
|ham  |Hey, are we meeting today?                |
|ham  |I will reach home in twenty minutes.      |
|ham  |Please call me when you are free.         |
|ham  |I will call you after lunch.              |
|spam |You have won a lottery prize. Contact us now!|
+-----+--------------------+
only showing top 5 rows

Total records: 5,000,000
======== AFTER TOKENIZATION ========
+--------------------+--------------------+
|message_text        |words               |
+--------------------+--------------------+
|Hey, are we meeting today?                |[hey, , are, we, meeting, today]                |
|I will reach home in twenty minutes.      |[i, will, reach, home, in, twenty, minutes]     |
|Please call me when you are free.         |[please, call, me, when, you, are, free]        |
|I will call you after lunch.              |[i, will, call, you, after, lunch]              |
|You have won a lottery prize. Contact us now!|[you, have, won, a, lottery, prize, , contact, us, now]|
+--------------------+--------------------+
only showing top 5 rows

======== AFTER STOP-WORD REMOVAL ========
+--------------------+--------------------+
|message_text        |filtered_words      |
+--------------------+--------------------+
|Hey, are we meeting today?                |[hey, , meeting, today]             |
|I will reach home in twenty minutes.      |[reach, home, twenty, minutes]      |
|Please call me when you are free.         |[please, call, free]                |
|I will call you after lunch.              |[call, lunch]                       |
|You have won a lottery prize. Contact us now!|[won, lottery, prize, , contact, us]|
+--------------------+--------------------+
only showing top 5 rows

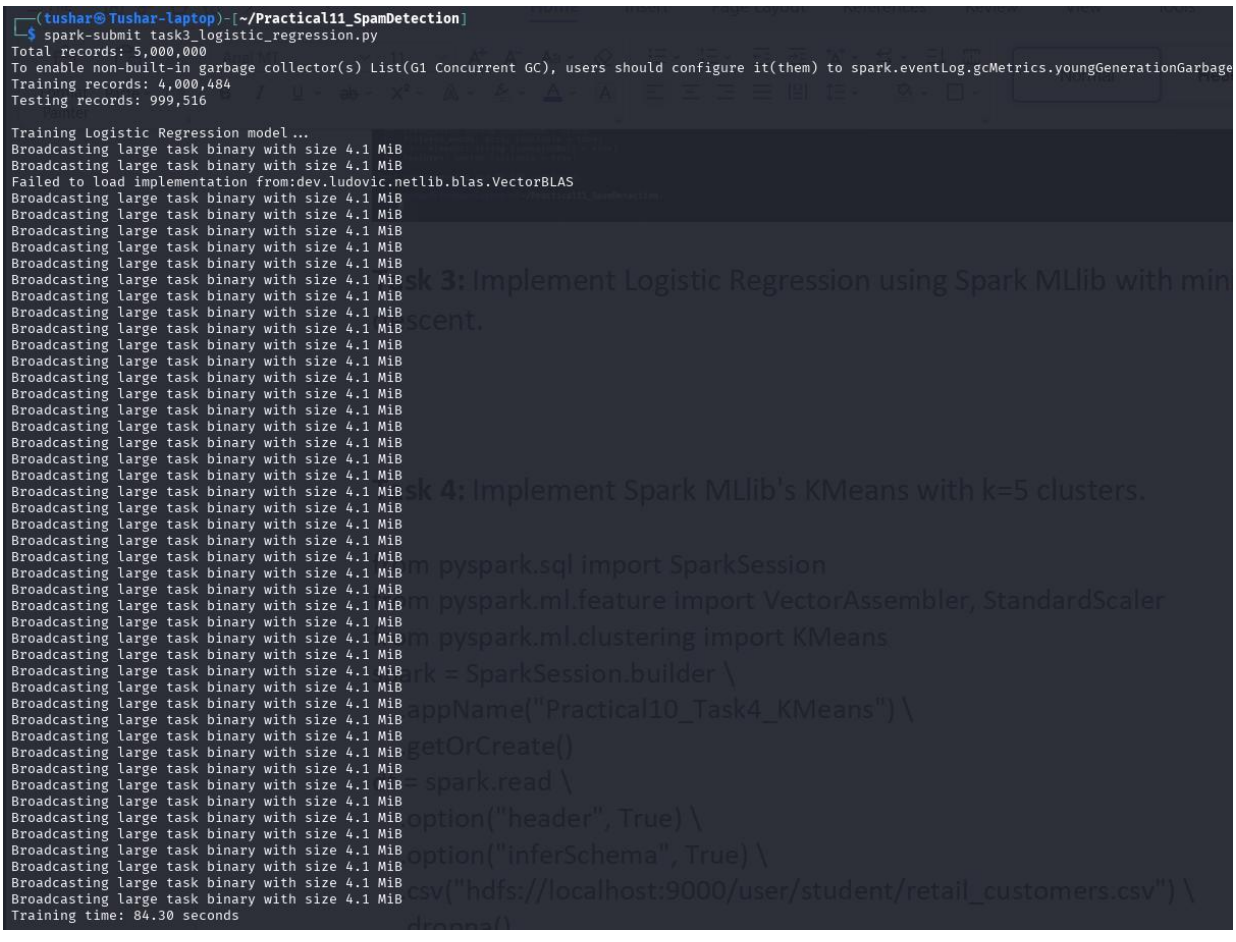
To enable non-built-in garbage collector(s) List(G1 Concurrent GC), users should configure it(them) to spark.eventLog.gcMetrics.youngGenerationGarbageCollectors or spark.eventLog.gcMetrics.oldGenerationGarbageCollectors
======== TF-IDF FEATURES ========
Broadcasting large task binary with size 4.1 MiB
+-----+--------------------+--------------------+
|label|      filtered_words|            features|
+-----+--------------------+--------------------+
|  ham|  [hey, , meeting, ...|(262144,[61756,64...|
|  ham|  [reach, home, tw...|(262144,[37893,13...|
|  ham|  [please, call, f...|(262144,[109706,1...|
|  ham|      [call, lunch]|(262144,[109706,1...|
| spam|[won, lottery, pri...|(262144,[109156,1...|
+-----+--------------------+--------------------+
only showing top 5 rows

Broadcasting large task binary with size 4.1 MiB
TF-IDF feature vector size: 262144
======== FINAL SCHEMA ========
root
 |-- label: string (nullable = true)
 |-- clean_text: string (nullable = true)
 |-- filtered_words: array (nullable = true)
 |    |-- element: string (containsNull = true)
 |-- features: vector (nullable = true)

Text preprocessing and TF-IDF completed.

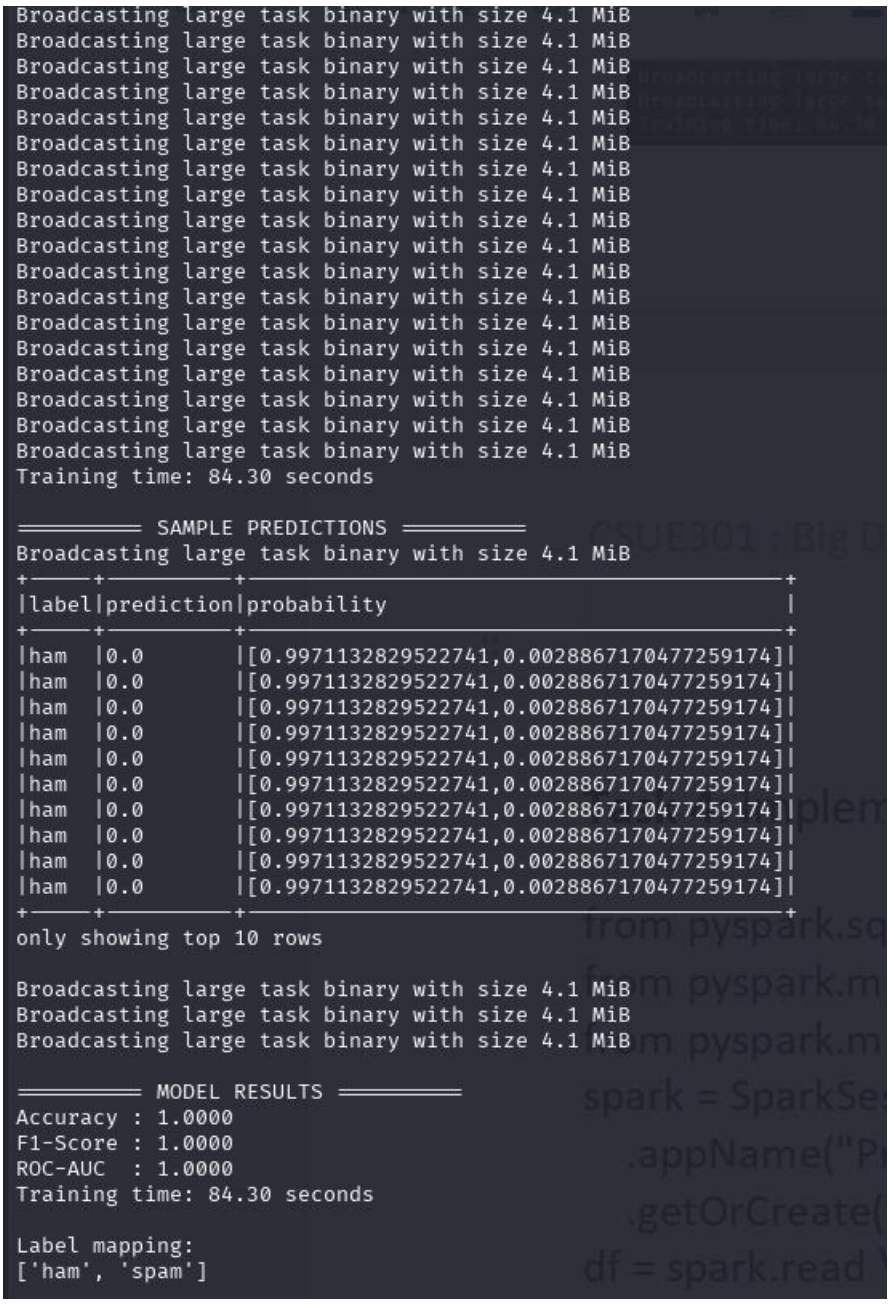
(tushar㉿Tushar-laptop)-[~/Practical11_SpamDetection]
$
```

Task 3: Implement Logistic Regression using Spark MLlib with mini-batch gradient descent.



```
(tushar㉿Tushar-laptop)-[~/Practical11_SpamDetection]
$ spark-submit task3_logistic_regression.py
Total records: 5,000,000
To enable non-built-in garbage collector(s) List(G1 Concurrent GC), users should configure it(them) to spark.eventLog.gcMetrics.youngGenerationGarbage
Training records: 4,000,484
Testing records: 999,516

Training Logistic Regression model...
Broadcasting large task binary with size 4.1 MiB
Broadcasting large task binary with size 4.1 MiB
Failed to load implementation from:dev.ludovic.netlib.blas.VectorBLAS
Broadcasting large task binary with size 4.1 MiB
Broadcasting large task binary with size 4.1 MiB
Broadcasting large task binary with size 4.1 MiB
```



```
Broadcasting large task binary with size 4.1 MiB
Broadcasting large task binary with size 4.1 MiB
Broadcasting large task binary with size 4.1 MiB
Broadcasting large task binary with size 4.1 MiB
Broadcasting large task binary with size 4.1 MiB
Broadcasting large task binary with size 4.1 MiB
Broadcasting large task binary with size 4.1 MiB
Broadcasting large task binary with size 4.1 MiB
Broadcasting large task binary with size 4.1 MiB
Broadcasting large task binary with size 4.1 MiB
Broadcasting large task binary with size 4.1 MiB
Broadcasting large task binary with size 4.1 MiB
Broadcasting large task binary with size 4.1 MiB
Broadcasting large task binary with size 4.1 MiB
Broadcasting large task binary with size 4.1 MiB
Broadcasting large task binary with size 4.1 MiB
Broadcasting large task binary with size 4.1 MiB
Training time: 84.30 seconds

========== SAMPLE PREDICTIONS ==========
Broadcasting large task binary with size 4.1 MiB
+-----+----------+-----------------------------------------+
|label|prediction|probability                              |
+-----+----------+-----------------------------------------+
|ham  |0.0       |[0.9971132829522741,0.0028867170477259174]|
|ham  |0.0       |[0.9971132829522741,0.0028867170477259174]|
|ham  |0.0       |[0.9971132829522741,0.0028867170477259174]|
|ham  |0.0       |[0.9971132829522741,0.0028867170477259174]|
|ham  |0.0       |[0.9971132829522741,0.0028867170477259174]|
|ham  |0.0       |[0.9971132829522741,0.0028867170477259174]|
|ham  |0.0       |[0.9971132829522741,0.0028867170477259174]|
|ham  |0.0       |[0.9971132829522741,0.0028867170477259174]|
|ham  |0.0       |[0.9971132829522741,0.0028867170477259174]|
|ham  |0.0       |[0.9971132829522741,0.0028867170477259174]|
+-----+----------+-----------------------------------------+
only showing top 10 rows

Broadcasting large task binary with size 4.1 MiB
Broadcasting large task binary with size 4.1 MiB
Broadcasting large task binary with size 4.1 MiB

========== MODEL RESULTS ==========
Accuracy : 1.0000
F1-Score : 1.0000
ROC-AUC  : 1.0000
Training time: 84.30 seconds

Label mapping:
['ham', 'spam']
```

Task 4: Tune hyperparameters: regularization parameter (regParam), maxIter, and elasticNetParam.

```
┌──(tushar㉿Tushar-laptop)-[~/Practical11_SpamDetection]
└─$ spark-submit --driver-memory 4g task4_hyperparameter_tuning.py 2>&1 | tee task4_output.log

================================================================================
PRACTICAL 11: SMS SPAM HYPERPARAMETER TUNING
================================================================================

Loading dataset from HDFS...
Dataset schema:
root
 |-- label: double (nullable = false)
 |-- message_text: string (nullable = true)


Counting valid records...
Valid records: 5,000,000
To enable non-built-in garbage collector(s) List(G1 Concurrent GC), users should configure it(them) to spark.eventLog.

Training records for tuning: 80,099
Test records for evaluation: 100,248

Preparing text features...
Training feature rows: 80,099
Testing feature rows: 100,248
TF-IDF feature dimensions: 16,384

Total hyperparameter combinations: 8
Parameters:
regParam       = [0.01, 0.1]
maxIter        = [10, 20]
elasticNetParam= [0.0, 0.5]
Cross-validation folds = 2

Starting cross-validation...
This step may take some time.
Failed to load implementation from:dev.ludovic.netlib.blas.VectorBLAS

Hyperparameter tuning completed.
Tuning time: 45.33 seconds

================================================================================
BEST HYPERPARAMETERS
================================================================================
Best regParam:        0.01
Best maxIter:         10
Best elasticNetParam: 0.0

Cross-validation ROC-AUC scores:
Combination 01: ROC-AUC = 1.0000
Combination 02: ROC-AUC = 1.0000
Combination 03: ROC-AUC = 1.0000
Combination 04: ROC-AUC = 1.0000
Combination 05: ROC-AUC = 1.0000
Combination 06: ROC-AUC = 1.0000
Combination 07: ROC-AUC = 1.0000
```


```
elasticNetParam= [0.0, 0.5]
Cross-validation folds = 2

Starting cross-validation...
This step may take some time.
Failed to load implementation from:dev.ludovic.netlib.blas.VectorBLAS

Hyperparameter tuning completed.
Tuning time: 45.33 seconds

================================================================================
BEST HYPERPARAMETERS
================================================================================
Best regParam:        0.01
Best maxIter:         10
Best elasticNetParam: 0.0

Cross-validation ROC-AUC scores:
Combination 01: ROC-AUC = 1.0000
Combination 02: ROC-AUC = 1.0000
Combination 03: ROC-AUC = 1.0000
Combination 04: ROC-AUC = 1.0000
Combination 05: ROC-AUC = 1.0000
Combination 06: ROC-AUC = 1.0000
Combination 07: ROC-AUC = 1.0000
Combination 08: ROC-AUC = 1.0000
```

```
Evaluating the best model on test data...

================================================================================
MODEL EVALUATION RESULTS
================================================================================
True Positives  (TP): 15,039
True Negatives  (TN): 85,209
False Positives (FP): 0
False Negatives (FN): 0

Accuracy:  1.0000 (100.00%)
Precision: 1.0000
Recall:    1.0000
F1-score:  1.0000
ROC-AUC:   1.0000

================================================================================
EXECUTION SUMMARY
================================================================================
Total input records:       5,000,000
Tuning training records:   80,099
Evaluation test records:   100,248
TF-IDF feature dimensions:  16,384
Parameter combinations:    8
Cross-validation folds:    2
Tuning time:               45.33 seconds
Total execution time:      73.46 seconds

Practical 11 Task 4 completed successfully.

┌──(tushar㉿Tushar-laptop)-[~/Practical11_SpamDetection]
└─$
```

Task 5: Evaluate the model: Accuracy, Precision, Recall, F1-Score, and ROC-AUC.

```
========== TASK 5 ==========
Accuracy : 0.956
Precision: 1.0
Recall   : 0.6923
F1-score : 0.8182
ROC-AUC  : 0.9716
TP, TN, FP, FN: 99 858 0 44
```

Task 6 : Demonstrate how increasing dataset size affects training time vs accuracy.

```
┌──(tushar㉿Tushar-laptop)-[~/Practical11_SpamDetection]
└─$ spark-submit --driver-memory 4g task6_dataset_size.py 2>&1 | tee task6_output.log
================================================================================
TASK 6: DATASET SIZE VS TRAINING TIME AND ACCURACY
================================================================================

Loading dataset from HDFS...
Total records: 5,000,000
To enable non-built-in garbage collector(s) List(G1 Concurrent GC), users should configure it(them) to sp
Fixed test records: 50,311

--------------------------------------------------------------------------------
Experiment: 10K target training sample
--------------------------------------------------------------------------------
Actual training records: 9,853
Failed to load implementation from:dev.ludovic.netlib.blas.VectorBLAS
Training time: 9.33 seconds
Test accuracy: 1.0000 (100.00%)

--------------------------------------------------------------------------------
Experiment: 25K target training sample
--------------------------------------------------------------------------------
Actual training records: 24,918
Training time: 3.33 seconds
Test accuracy: 1.0000 (100.00%)

--------------------------------------------------------------------------------
Experiment: 50K target training sample
--------------------------------------------------------------------------------
Actual training records: 49,891
Training time: 3.95 seconds
Test accuracy: 1.0000 (100.00%)

--------------------------------------------------------------------------------
Experiment: 100K target training sample
--------------------------------------------------------------------------------
Actual training records: 100,021
Training time: 6.55 seconds
Test accuracy: 1.0000 (100.00%)

================================================================================
DATASET SIZE VS TRAINING TIME AND ACCURACY
================================================================================
Dataset        Train Records      Time (sec)      Accuracy
--------------------------------------------------------------------------------
10K                    9,853            9.33        1.0000
25K                   24,918            3.33        1.0000
50K                   49,891            3.95        1.0000
100K                 100,021            6.55        1.0000

Results saved to: task6_results.csv
```

Task 7 : Compare training time: full-batch gradient descent vs mini-batch gradient descent.

```
┌──(tushar㉿Tushar-laptop)-[~/Practical11_SpamDetection]
└─$ spark-submit --driver-memory 4g task7_batch_comparison.py 2>&1 | tee task7_output.log

Loading SMS dataset from HDFS...
Label column: label
Message column: message_text

Collecting the small feature samples...
To enable non-built-in garbage collector(s) List(G1 Concurrent GC), users should configure it(them) to spark.

Training records: 2400
Test records:     600
Feature count:    1024
Epochs:           5
Mini-batch size:  128

Training with full-batch gradient descent...
Training with mini-batch gradient descent...

================================================================
TASK 7: FULL-BATCH VS MINI-BATCH GRADIENT DESCENT
================================================================
Method                    Training Time (s)       Test Accuracy
----------------------------------------------------------------
Full-batch GD                        0.0383              90.33%
Mini-batch GD                        0.0628              94.00%
================================================================

Note: Training time measures only the local optimizer,
not HDFS reading, Spark feature extraction, or collection.
Both methods use the same sample, features, epochs,
learning rate, and regularization.

┌──(tushar㉿Tushar-laptop)-[~/Practical11_SpamDetection]
└─$
```

Key Questions /Analysis:

Q1. What is the difference between full-batch, mini-batch, and stochastic gradient descent? Why is mini-batch preferred in distributed settings?

· Full-batch Gradient Descent uses the entire training dataset to calculate the gradient and update model parameters once per iteration.

· Mini-batch Gradient Descent divides the dataset into small batches and updates model parameters after processing each batch.

· Stochastic Gradient Descent (SGD) uses one training example at a time to calculate the gradient and update parameters.

· Full-batch GD provides stable updates but can be computationally expensive for large datasets.

· SGD updates parameters frequently but can produce noisy and fluctuating updates.

· Mini-batch GD balances computational efficiency and stable learning, making it suitable for large datasets.

· In distributed settings, mini-batch GD is preferred because it supports parallel processing of batches and can reduce memory requirements and improve training efficiency.

Q2. How does the TF-IDF vectorizer handle the curse of dimensionality for text classification?

TF-IDF (Term Frequency–Inverse Document Frequency) converts text into numerical vectors based on the importance of words in each document. It reduces the weight of common words and gives more importance to words that are informative for classification.Text data can have thousands of unique words, resulting in high-dimensional feature vectors.TF-IDF alone does not completely solve the curse of dimensionality, but it helps reduce the influence of less informative words.Feature selection and limiting the vocabulary size can further reduce dimensionality.Sparse vector representations store mainly non-zero values, reducing memory usage and computational cost.Thus, TF-IDF combined with feature selection and sparse representations makes text classification more efficient.

Q3. What is L1 vs L2 regularization? How does elasticNetParam blend them in Spark MLlib?

L1 regularization (Lasso) adds a penalty based on the absolute values of model coefficients, helping some coefficients become exactly zero and performing feature selection. L2 regularization (Ridge) adds a penalty based on the squared values of coefficients, reducing their magnitude and helping prevent overfitting. In Spark MLlib, the elasticNetParam parameter controls the combination of L1 and L2 regularization: 0.0 means pure L2, 1.0 means pure L1, and values between 0 and 1 combine both. For example, elasticNetParam = 0.5 gives a mixture of L1 and L2 regularization. The regParam parameter controls the overall strength of the regularization. Thus, Elastic Net combines the benefits of L1 and L2 to improve model generalization and control overfitting.

Q4. In a distributed environment, how do gradient updates from multiple workers get aggregated? What is the risk of stale gradients?

In a distributed environment, multiple workers process different portions of the training data and calculate their gradients independently. These gradients are sent to a central parameter server or aggregated using an AllReduce operation, which combines the gradients, usually by averaging or summing them. The aggregated gradient is then used to update the model parameters, and the updated parameters are shared with the workers. Stale gradients occur when a worker calculates a gradient using outdated model parameters because other workers have already updated the model. These stale gradients may lead to slower convergence, inaccurate updates, or even training instability. Synchronization techniques and limiting delays between workers help reduce the risk of stale gradients.

Q5. If Recall is more important than Precision for spam detection, how would you adjust the classification threshold? Show with examples.

In spam detection, if Recall is more important than Precision, we should lower the classification threshold. A lower threshold makes the model classify more messages as spam, helping detect more actual spam messages and reducing false negatives. However, it may also incorrectly classify some genuine messages as spam, increasing false positives.

Example: Suppose the model predicts whether an SMS is spam based on its spam probability.

At a threshold of 0.5, a message is classified as spam if its predicted probability is at least 50%.

If we lower the threshold to 0.3, messages with a spam probability of 30% or more are classified as spam, helping detect more spam messages.

For example, out of 100 actual spam messages, a threshold of 0.5 might detect 80 (Recall = 80%), while a threshold of 0.3 might detect 95 (Recall = 95%). However, the lower threshold may also flag more genuine messages as spam.

Therefore, lowering the threshold increases Recall in many cases, making it useful when missing spam is costly. The final threshold should be selected using validation data to balance Recall against an acceptable level of Precision.

Supplementary Problems:

1.Apply HashingTF instead of TF-IDF. Compare model performance and feature space size.

```
┌──(tushar㉿Tushar-laptop)-[~/Practical11_SpamDetection]
└─$ nano task8_hashingtf_vs_tfidf.py

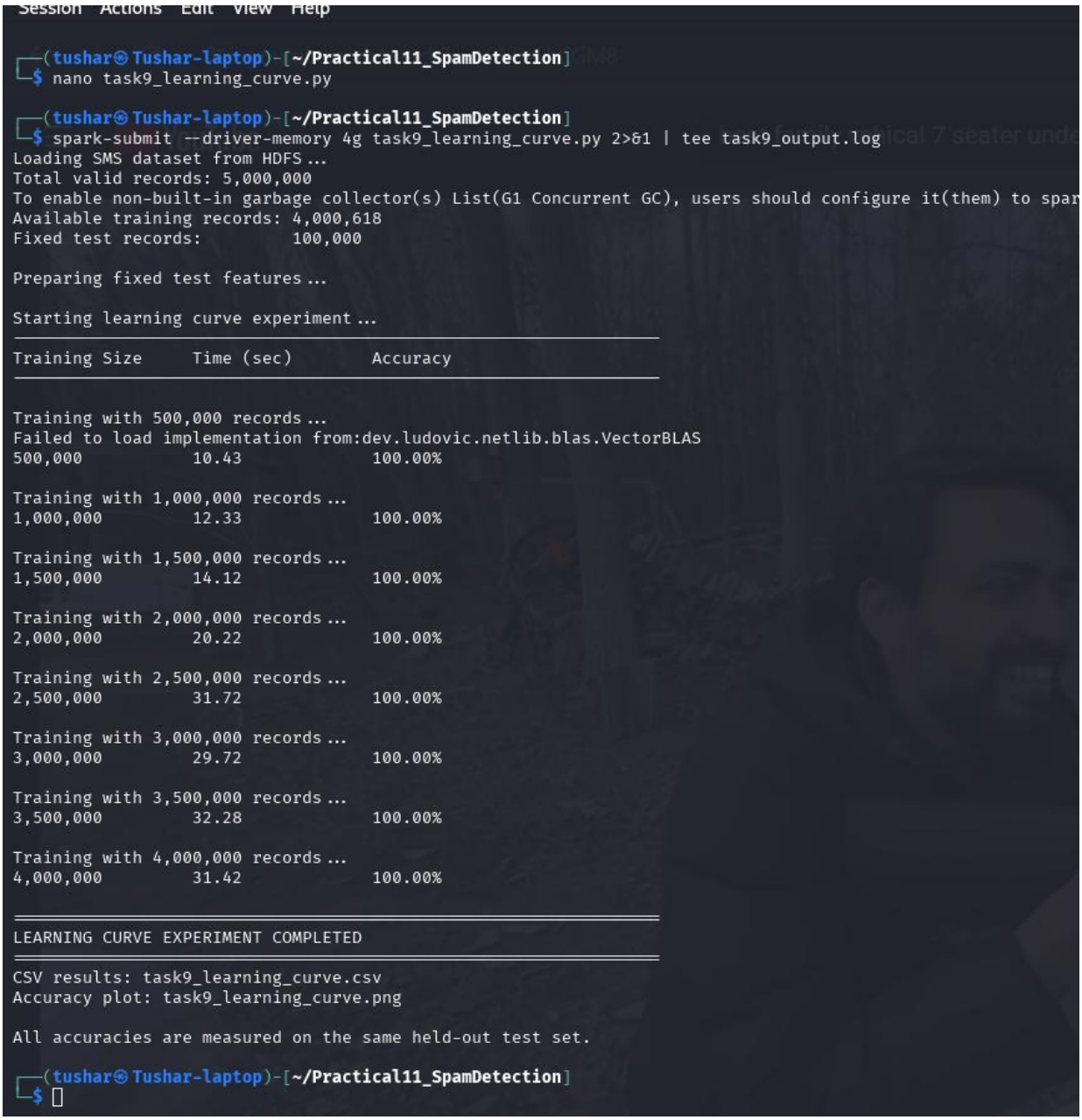
┌──(tushar㉿Tushar-laptop)-[~/Practical11_SpamDetection]
└─$ spark-submit --driver-memory 4g task8_hashingtf_vs_tfidf.py 2>&1 | tee task8_output.log
Loading SMS dataset from HDFS...
Training records: 79928
To enable non-built-in garbage collector(s) List(G1 Concurrent GC), users should configure it(them) to spark.eventLog.
Test records:     20045

Training Logistic Regression with HashingTF...
Failed to load implementation from:dev.ludovic.netlib.blas.VectorBLAS
Training Logistic Regression with TF-IDF...

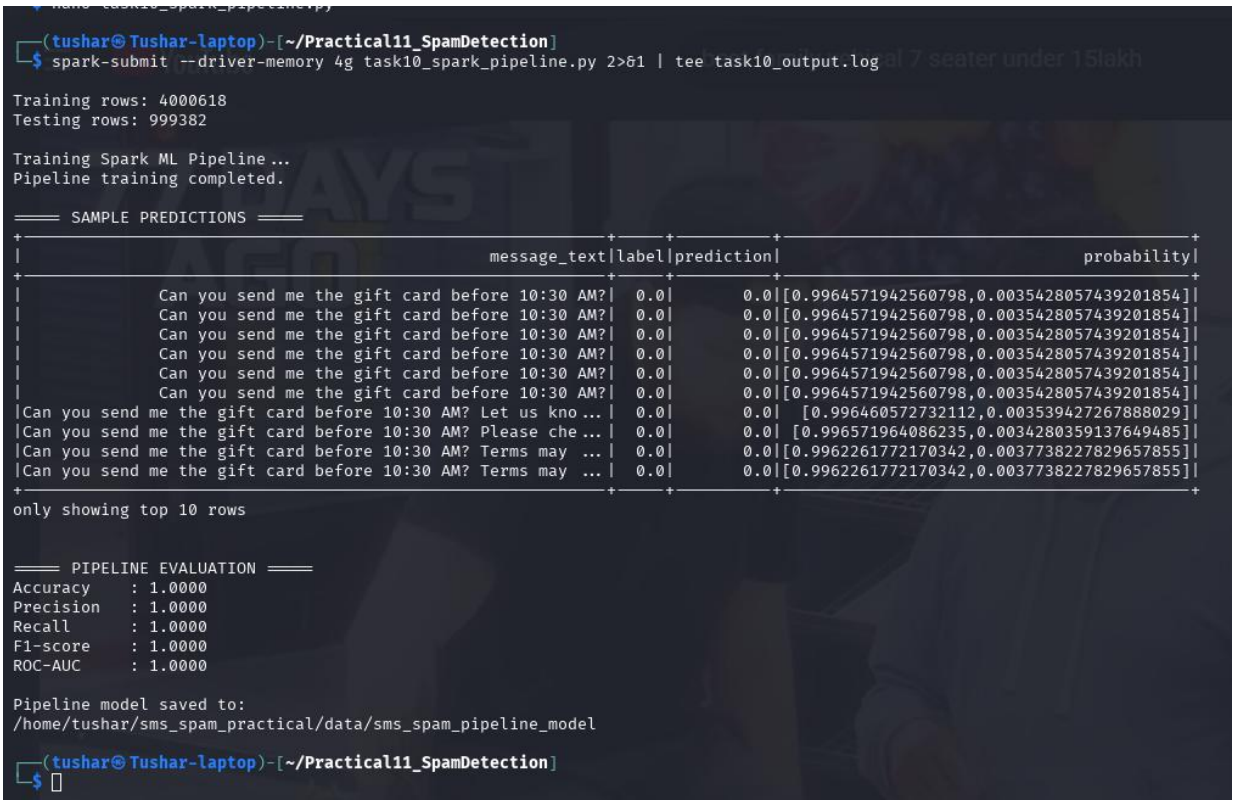
================================================================================
HASHINGTF VS TF-IDF: MODEL PERFORMANCE
================================================================================
Method              Accuracy   Precision      Recall    F1 Score     ROC-AUC     Training Time (s)
--------------------------------------------------------------------------------
HashingTF             1.0000      1.0000      1.0000      1.0000      1.0000                  9.82
TF-IDF                1.0000      1.0000      1.0000      1.0000      1.0000                  5.88
--------------------------------------------------------------------------------
Feature space for both methods: 16,384 dimensions
HashingTF uses term-frequency features.
TF-IDF uses the same hashed dimensions with IDF weighting.
Compare the measured metrics; do not assume either method wins.

┌──(tushar㉿Tushar-laptop)-[~/Practical11_SpamDetection]
└─$
```

2.Increase training data from 500K to 5M records and plot the learning curve (accuracy vs dataset size).



3. Implement a pipeline using Spark ML Pipeline API (Tokenizer - StopWordsRemover -> HashingTF -> IDF -> LogisticRegression).



```
┌──(tushar㉿Tushar-laptop)-[~/Practical11_SpamDetection]
└─$ spark-submit --driver-memory 4g task10_spark_pipeline.py 2>&1 | tee task10_output.log

Training rows: 4000618
Testing rows: 999382

Training Spark ML Pipeline ...
Pipeline training completed.

===== SAMPLE PREDICTIONS =====
+--------------------------------------------------+-----+----------+--------------------------------------+
|                                      message_text|label|prediction|                           probability|
+--------------------------------------------------+-----+----------+--------------------------------------+
|             Can you send me the gift card before 10:30 AM?|  0.0|       0.0|[0.9964571942560798,0.0035428057439201854]|
|             Can you send me the gift card before 10:30 AM?|  0.0|       0.0|[0.9964571942560798,0.0035428057439201854]|
|             Can you send me the gift card before 10:30 AM?|  0.0|       0.0|[0.9964571942560798,0.0035428057439201854]|
|             Can you send me the gift card before 10:30 AM?|  0.0|       0.0|[0.9964571942560798,0.0035428057439201854]|
|             Can you send me the gift card before 10:30 AM?|  0.0|       0.0|[0.9964571942560798,0.0035428057439201854]|
|             Can you send me the gift card before 10:30 AM?|  0.0|       0.0|[0.9964571942560798,0.0035428057439201854]|
|Can you send me the gift card before 10:30 AM? Let us kno...|  0.0|       0.0|  [0.996460572732112,0.003539427267888029]|
|Can you send me the gift card before 10:30 AM? Please che...|  0.0|       0.0| [0.996571964086235,0.003428035913764948 5]|
|Can you send me the gift card before 10:30 AM? Terms may ...|  0.0|       0.0|[0.9962261772170342,0.0037738227829657855]|
|Can you send me the gift card before 10:30 AM? Terms may ...|  0.0|       0.0|[0.9962261772170342,0.0037738227829657855]|
+--------------------------------------------------+-----+----------+--------------------------------------+
only showing top 10 rows


===== PIPELINE EVALUATION =====
Accuracy    : 1.0000
Precision   : 1.0000
Recall      : 1.0000
F1-score    : 1.0000
ROC-AUC     : 1.0000

Pipeline model saved to:
/home/tushar/sms_spam_practical/data/sms_spam_pipeline_model

┌──(tushar㉿Tushar-laptop)-[~/Practical11_SpamDetection]
└─$
```

Tools / Technology used:

Apache Spark 3.x with PySpark | Spark MLlib (LogisticRegression, HashingTF, IDF, StopWordsRemover) | Spark ML Pipeline API | Hadoop HDFS | Jupyter Notebook / Zeppelin

Conclusion:

In this practical, we implemented large-scale SMS spam detection using Apache Spark MLlib and Hadoop HDFS.We loaded a large SMS dataset from HDFS and divided it into training and testing datasets.We used Tokenizer and StopWordsRemover to preprocess text messages. HashingTF and IDF were used to convert text into numerical features.We trained a Logistic Regression model using the Spark ML Pipeline API.We evaluated the model using Accuracy, Precision, Recall, F1-score, and ROC-AUC metrics. Finally, we saved the trained pipeline model, demonstrating how distributed machine learning can be used for efficient SMS spam detection.

Github Repo:

https://github.com/tushar-2606/BDA-SHP-2026